

# Day 24: Calculate Total Deposits

## 1. Day Number

Day 24

## 2. Topic Name

Accumulator Pattern

## 3. Connection

Earlier, you saved deposit transactions in a list. Today, you will go through those deposits and calculate the **total deposited amount**.

The accumulator pattern is useful whenever you need to combine several values into one result.

---

## 4. Important Topics

- List of numbers
  - for loop
  - Accumulator variable
  - Updating a running total
  - Function parameter
  - return value
- 

## 5. Foundational Notes

### What is an accumulator?

An accumulator is a variable that stores a result while a loop is running.

For addition, it normally begins with 0.

```
total = 0
```

Each number is then added to the current total:

```
total = total + number
```

You can also write the same operation using +=:

```
total += number
```

### How the total changes

Suppose the deposits are:

```
[100, 250, 50]
```

The calculation happens like this:

| Current deposit | Total before | Total after |
|-----------------|--------------|-------------|
| \$100           | \$0          | \$100       |
| \$250           | \$100        | \$350       |
| \$50            | \$350        | \$400       |

Final total:

\$400

### Why does the total start at 0?

Because initially, no deposit has been added.

Starting with another number would produce an incorrect result.

### What happens with an empty list?

When the list is empty, the loop runs zero times. The accumulator remains 0.

---

## 6. Easy Example

This example adds some simple scores:

```
scores = [5, 10, 15]
total_score = 0

for score in scores:
    total_score += score

print(total_score)
```

Output:

30

Here:

- total\_score is the accumulator.
  - score represents one value from the list.
  - Each score is added to the running total.
- 

## 7. Problem Statement

Create a list containing several deposit amounts.

Write a function that:

1. Receives the deposit list.
2. Creates an accumulator variable.
3. Uses a loop to visit every deposit.
4. Adds each deposit to the accumulator.
5. Returns the total deposit amount.

Call the function and print the returned total.

Example deposit list:

```
[500, 1200, 300]
```

Expected total:

```
$2000
```

---

## 8. Concepts Used

### List

A list stores multiple deposit amounts together.

```
deposits = [500, 1200, 300]
```

### Function

A function organizes the calculation into a reusable block.

```
def calculate_something(values):
```

### Parameter

The deposit list can be passed into the function as a parameter.

### Accumulator variable

A variable begins at 0 and stores the running total.

### for loop

The loop processes each deposit one by one.

### return

The function sends the final total back to the place where it was called.

---

## 9. Thought Process

Think about the problem in these steps:

### Step 1: Store the deposits

Create a list of numeric deposit amounts.

### Step 2: Create a function

The function should receive the list as its parameter.

### Step 3: Start the total at zero

Before processing the deposits, nothing has been added.

```
total = 0
```

#### Step 4: Visit each deposit

Use a for loop to read one amount at a time.

#### Step 5: Update the accumulator

Add the current deposit to the existing total.

```
new total = old total + current deposit
```

#### Step 6: Return the result

After the loop finishes, return the total.

#### Step 7: Print the returned value

Call the function and display the result outside the function.

---

### 10. Pseudocode

```
FUNCTION calculate_total_deposits(deposit_list)

    SET total to 0

    FOR each deposit in deposit_list
        ADD deposit to total
    END FOR

    RETURN total

END FUNCTION

CREATE a list of deposit amounts

CALL calculate_total_deposits with the list
STORE the returned result

PRINT the total deposit amount
```

### 11. Suggested Solving Approach

Use a functional approach:

```
Deposit list
  ↓
Pass list to function
  ↓
Function loops through deposits
  ↓
Accumulator calculates total
  ↓
Function returns total
  ↓
Print returned total
```

Suggested structure:

```
def calculate_total_deposits(deposits):
    # Create accumulator
    # Loop through deposits
    # Update accumulator
    # Return total

<div class="source-blank-line"></div>

deposit_amounts = [...]
result = calculate_total_deposits(deposit_amounts)
# Print result
```

Complete the missing logic yourself.

---

## 12. Easy Edge Cases

### Empty list

Input:

```
[]
```

Expected total:

```
$0
```

Because there are no deposits to add.

### Deposit amount is 0

Input:

```
[500, 0, 200]
```

Expected total:

```
$700
```

Adding zero does not change the total.

### All amounts are 0

Input:

```
[0, 0, 0]
```

Expected total:

```
$0
```

---

## 13. Expected Input

For this practice problem, you can create the list directly instead of asking the user for input.

Example:

```
deposit_amounts = [500, 1200, 300]
```

The function receives this list.

---

## 14. Expected Output

For this deposit list:

```
[500, 1200, 300]
```

The output could look like:

```
Total deposits: $2000
```

For an empty list:

```
Total deposits: $0
```

---

## 15. Hint Only

Inside your function:

1. Create a variable starting at 0.
2. Loop through the deposit list.
3. Use += to add each amount.
4. Return the accumulator **after** the loop finishes.

Be careful not to write `return` inside the loop. Otherwise, the function will stop after processing only the first deposit.